C PROGRAMMING ASSIGNMENT

ATM Transaction Simulation

Using Do-While Loop and Switch

Documentation Report

Submitted by	Vishal Prajapati
Roll Number	FDAI044-A
Subject	Programming with C
Faculty In-charge	Dr. Kishor Mahajan
Class	FY-BSC AI
Date	25 August 2026

Contents

1. Assignment Overview

This project simulates basic ATM transactions through a menu-driven console program. It provides balance enquiry, deposit, withdrawal, and exit operations. A do-while loop keeps the menu active, while switch selects the requested transaction and validation rules protect the account balance from invalid updates.

Item	Description
Assignment	Assignment 2 — ATM Transaction Simulation
Application type	Menu-driven console application
Primary C concepts	do-while loop, switch statement, variables, formatted I/O, relational operators, transaction validation, state update
Execution model	Interactive input, validation, operation, and repeated menu processing
Compiler command	gcc source_file.c

2. Problem Statement and Objectives

- Use a do-while loop so the ATM menu is displayed at least once.
- Use switch to select balance enquiry, deposit, withdrawal, or exit.
- Maintain and update an account balance.
- Reject zero and negative deposit or withdrawal amounts.
- Prevent withdrawals that exceed the available balance.
- Continue until the user explicitly selects option 4.

3. Initial Assumption and Menu

For demonstration, the account begins with an initial balance of Rs. 10,000.00. This value is stored in the balance variable and changes only after successful deposits or withdrawals.

Menu option	Operation	Effect on balance
1	Balance enquiry	No change; display current balance
2	Deposit	balance = balance + amount
3	Withdrawal	balance = balance - amount only if funds are sufficient
4	Exit	End the program

4. Algorithm

1. Set the initial balance to Rs. 10,000.00.
2. Display the ATM menu and read the user’s choice.
3. Use switch(choice) to execute the selected transaction.
4. For balance enquiry, display the current balance.
5. For deposit, accept the amount only if it is greater than zero, then add it to the balance.

6. For withdrawal, accept the amount only if it is greater than zero and does not exceed the balance. If valid, subtract it from the balance.
7. For an invalid menu choice, display an error message.
8. Repeat the process while choice is not 4.

5. Program Logic and Flowchart

The program initializes balance and enters a do-while loop. The menu is displayed before the loop condition is tested, which guarantees at least one display. The switch statement routes control to enquiry, deposit, withdrawal, or exit. Deposit and withdrawal amounts must be positive. A withdrawal is processed only when the requested amount does not exceed the current balance; otherwise, the transaction is declined and the balance remains unchanged.

Flowchart symbol key
Rounded rectangle = Start/End; parallelogram = Input/Output; rectangle = Processing; diamond = Decision; circle = Connector.

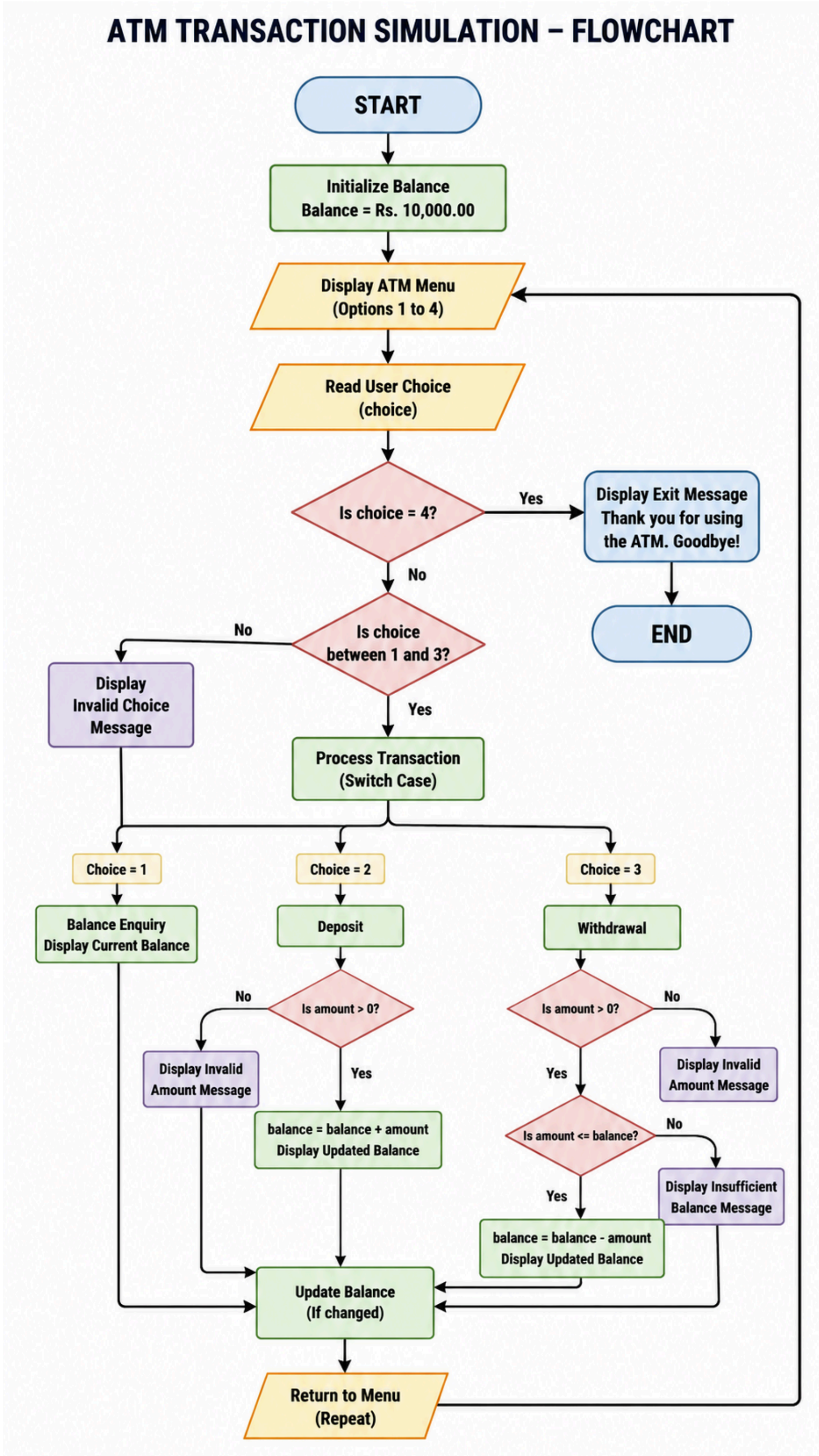

Figure 1. ATM flowchart showing menu repetition, transaction validation, and balance updates.

6. Transaction Validation and Loop Choice

A deposit or withdrawal amount must be greater than zero. Zero and negative values are rejected because they do not represent valid transactions. Before a withdrawal is processed, the program compares amount with balance. If amount is greater than balance, the transaction is declined and the account state remains unchanged.

Why a do-while loop is used
An ATM must display its menu and request at least one transaction before it can determine whether the user wants to exit. The do-while body executes first and tests the condition afterward, directly matching the requirement that the menu continue until Exit is selected.

7. Source Code

```
#include <stdio.h>

int main(void) {
    int choice;
    double balance = 10000.00;
    double amount;

    do {
        printf("\n===== ATM Transaction Simulation =====\n");
        printf("1. Balance Enquiry\n");
        printf("2. Deposit\n");
        printf("3. Withdrawal\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Please enter a number from 1 to 4.\n");
            while (getchar() != '\n') {
                /* Clear invalid input from the keyboard buffer. */
            }
            continue;
        }

        switch (choice) {
            case 1:
                printf("Available balance = Rs. %.2f\n", balance);
                break;

            case 2:
                printf("Enter deposit amount: Rs. ");
                if (scanf("%lf", &amount) != 1) {
                    printf("Invalid amount. Please enter a numeric value.\n");
                    while (getchar() != '\n') {
                        /* Clear invalid input from the keyboard buffer. */
                    }
                } else if (amount <= 0) {
                    printf("Invalid amount. Deposit must be greater than zero.\n");
                } else {
                    balance += amount;
                    printf("Deposit successful.\n");
                    printf("Updated balance = Rs. %.2f\n", balance);
                }
                break;

            case 3:
                printf("Enter withdrawal amount: Rs. ");
                if (scanf("%lf", &amount) != 1) {
                    printf("Invalid amount. Please enter a numeric value.\n");
                    while (getchar() != '\n') {
                        /* Clear invalid input from the keyboard buffer. */
                    }
```

```c
                } else if (amount <= 0) {
                    printf("Invalid amount. Withdrawal must be greater than zero.\n");
                } else if (amount > balance) {
                    printf("Transaction declined: Insufficient balance.\n");
                    printf("Available balance = Rs. %.2f\n", balance);
                } else {
                    balance -= amount;
                    printf("Please collect your cash.\n");
                    printf("Updated balance = Rs. %.2f\n", balance);
                }
                break;

            case 4:
                printf("Thank you for using the ATM. Goodbye!\n");
                break;

            default:
                printf("Invalid transaction choice. Please choose a number from 1 to 4.\n");
        }
    } while (choice != 4);

    return 0;
}
```

8. Test Cases and Verified Output

The following tests were executed after compilation with GCC. The sequence demonstrates the initial balance, a successful deposit, an excessive withdrawal, a successful withdrawal, invalid amount handling, invalid choice handling, and exit.

Test	Input sequence	Observed / expected output	Purpose
1	Choice 1	Rs. 10,000.00	Initial balance enquiry
2	Choice 2; Rs. 500	Balance becomes Rs. 10,500.00	Successful deposit
3	Choice 3; Rs. 20,000	Insufficient balance message	Reject excessive withdrawal
4	Choice 3; Rs. 1,000	Balance becomes Rs. 9,500.00	Successful withdrawal after deposit
5	Choice 2; Rs. -100	Invalid amount message	Reject negative deposit
6	Choice 5	Invalid transaction choice	Default case

```
atm_simulation — output evidence

$ verified run (GCC)
===== ATM Transaction Simulation =====
1. Balance Enquiry   2. Deposit   3. Withdrawal   4. Exit
Enter your choice: 1
Available balance = Rs. 10000.00

Enter your choice: 2
Enter deposit amount: Rs. 500
Deposit successful.
Updated balance = Rs. 10500.00

Enter your choice: 3
Enter withdrawal amount: Rs. 20000
Transaction declined: Insufficient balance.
Available balance = Rs. 10500.00

Enter your choice: 3
Enter withdrawal amount: Rs. 1000
Please collect your cash.
Updated balance = Rs. 9500.00

Enter your choice: 2
Enter deposit amount: Rs. -100
Invalid amount. Deposit must be greater than zero.

Enter your choice: 5
Invalid transaction choice. Please choose a number from 1 to 4.

Enter your choice: 4
Thank you for using the ATM. Goodbye!
```

Figure 2. Verified console output showing ATM transactions, validation, and final balances.

9. Conclusion

The ATM simulation uses the required do-while loop and switch statement. It supports balance enquiry, deposits, withdrawals, and exit. Its validation rules prevent invalid amounts and withdrawals that exceed the available balance. The verified console run confirms that successful transactions update the balance while declined or invalid transactions leave it unchanged.

10. References

[1] ISO C standard library reference, cppreference.com: https://en.cppreference.com/w/c

[2] C switch statement reference, cppreference.com: https://en.cppreference.com/w/c/language/switch

[3] C do-while loop reference, cppreference.com: https://en.cppreference.com/w/c/language/do

[4] C mathematical functions reference, cppreference.com: https://en.cppreference.com/w/c/numeric/math